REACH

Reinforcement-learning
for Enhanced Arm Control
in Humans

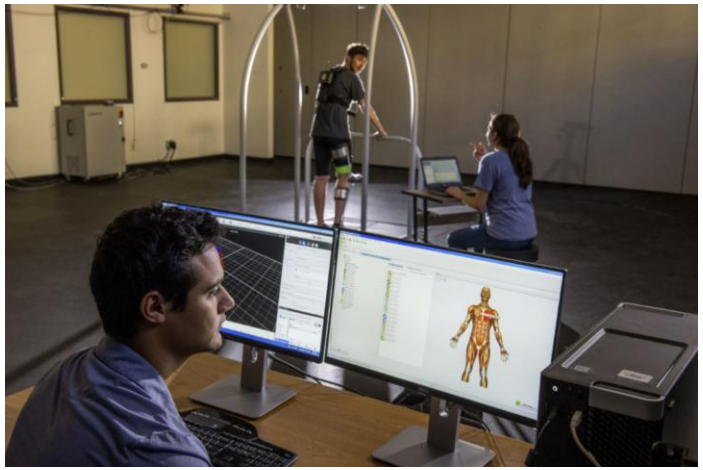
Members: TJ Davis, Lucas Larson, Clayton Ramsey, Victor Rodriguez

Clients: Dr. Zach Lerner & Dr. Carlo R. da Cunha

Mentor: Bailey Hall

Why REACH - Context, Need, and Vision

Biomechatronics Laboratory



NAU

Domain: Simulation and RL for robotics

Stakeholders: NAU Biomechatronics Lab - reproducible training & evaluation across tasks

Who: Biomechatronics Lab - Dr. Zach Lerner, Associate Professor, Mechanical Engineering, NAU & Dr Carlo Da Cunha, Assistant Professor, Electrical Engineering, NAU

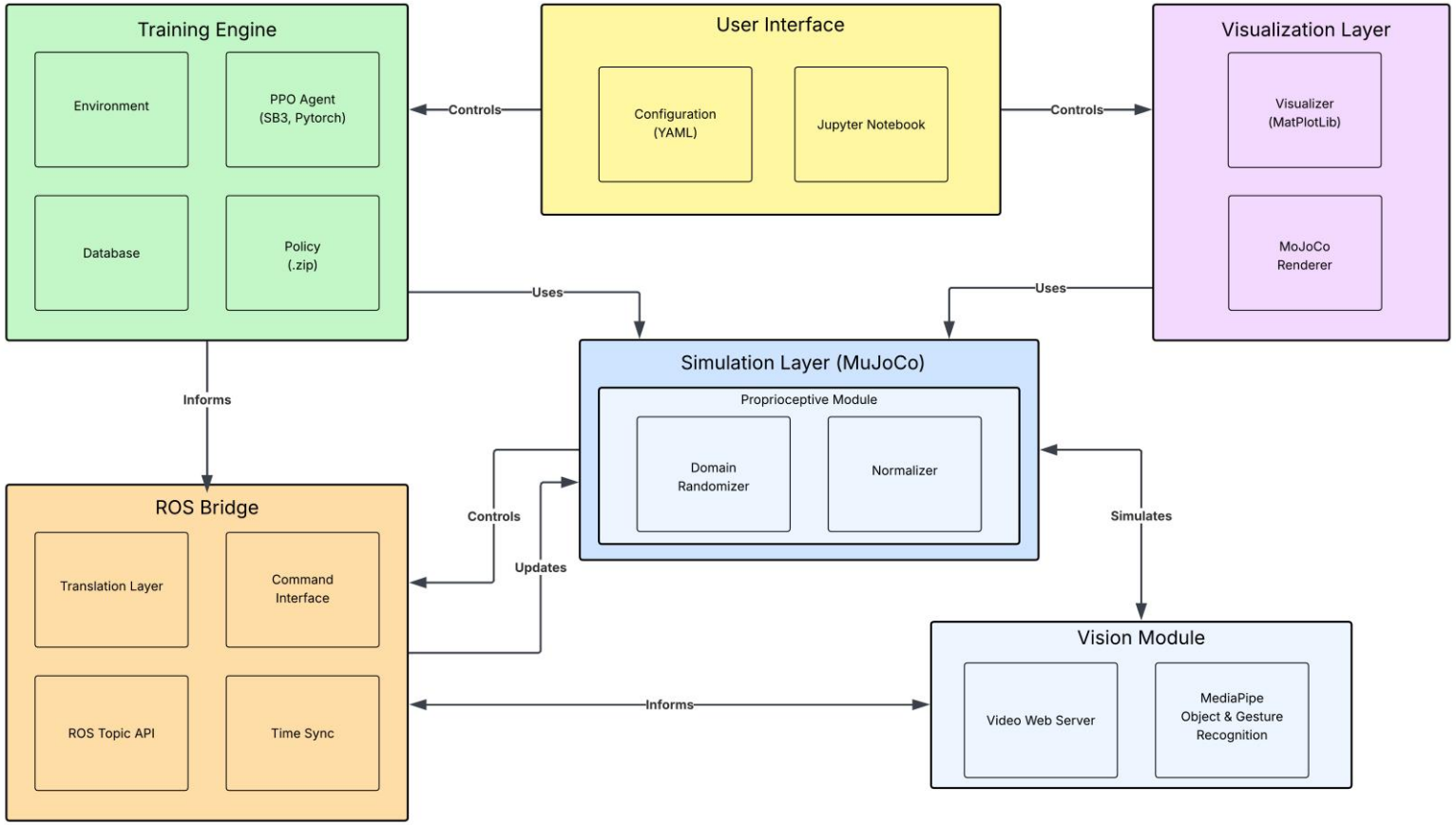
Gap: Fragmented, ad-hoc pipelines - hard to switch scenes, models, and policies

Specific needs: Reproducible training and evaluation, unified workflow to switch scenes, models, and policies, evidence for analysis, cross-platform support

Vision: REACH - one YAML driven workflow: train, evaluate, render, swap robots/scenes without rebuilding the stack

Software Architecture Overview

Tech Stack: Python • MuJoCo • Gymnasium • Stable-Baselines3 (PPO) • PyTorch • YAML



Testing Plan

UNIT TESTS: CONFIGURATION LOADER, ENVIRONMENT SETUP, AND CORE UTILITIES

INTEGRATION TESTS: FULL PIPELINE (CONFIGURATION → ENVIRONMENT → MUJOCO → PPO → FINAL OUTPUTS)

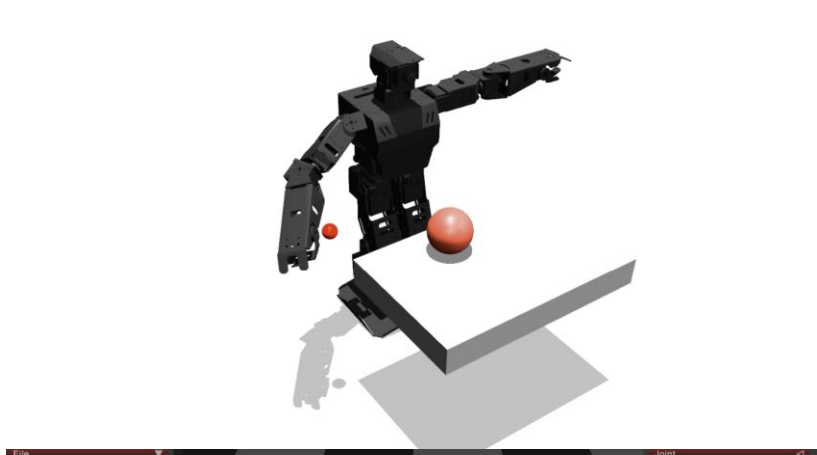
USABILITY TESTS: DEVELOPER WORKFLOWS AND USE-CASE READINESS

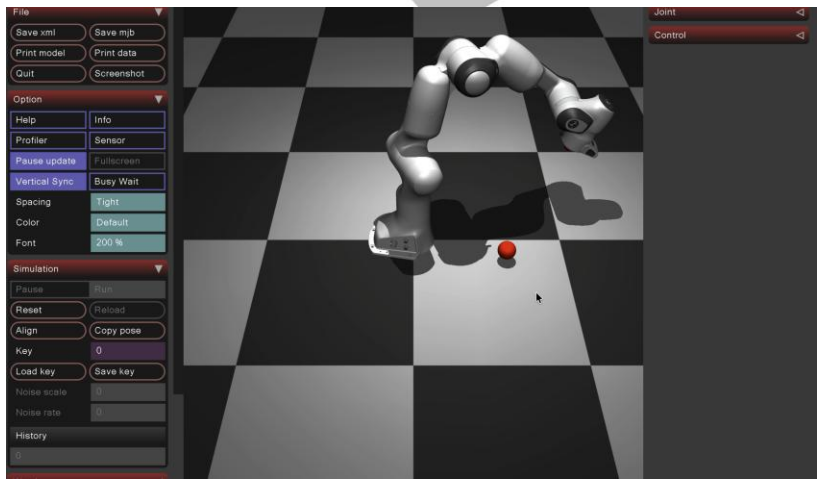
CONTINUOUS TESTING IN DEVELOPMENT CYCLE

LOGS, TEST RESULTS, AND CLIENT FEEDBACK GUIDE FIXES AND PRIORITIES

Design Phase Features

- Gesture Recognition Initialization
- Framework Adaptability
- Enhanced RL Reaching





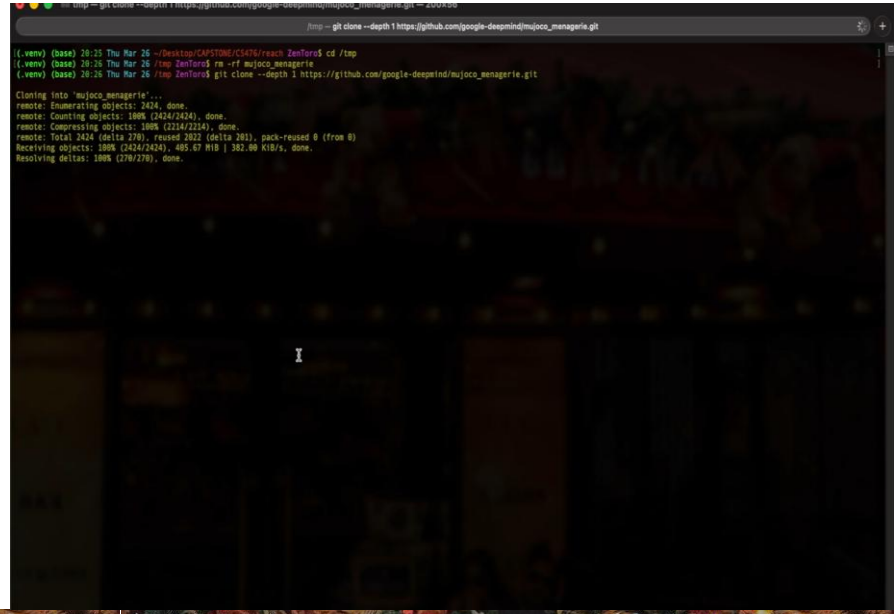
Gesture Recognition

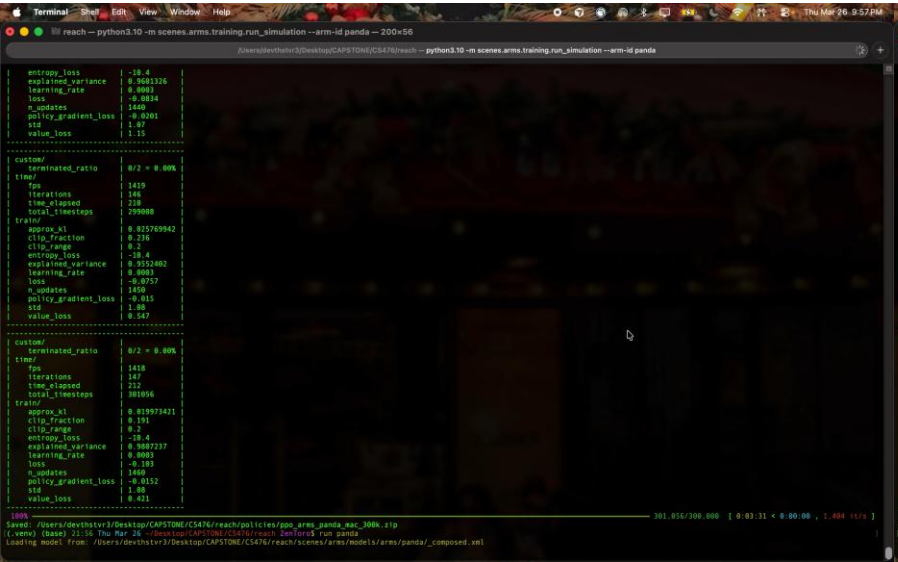
Multi-Robot, Multi-Scene Adaptability

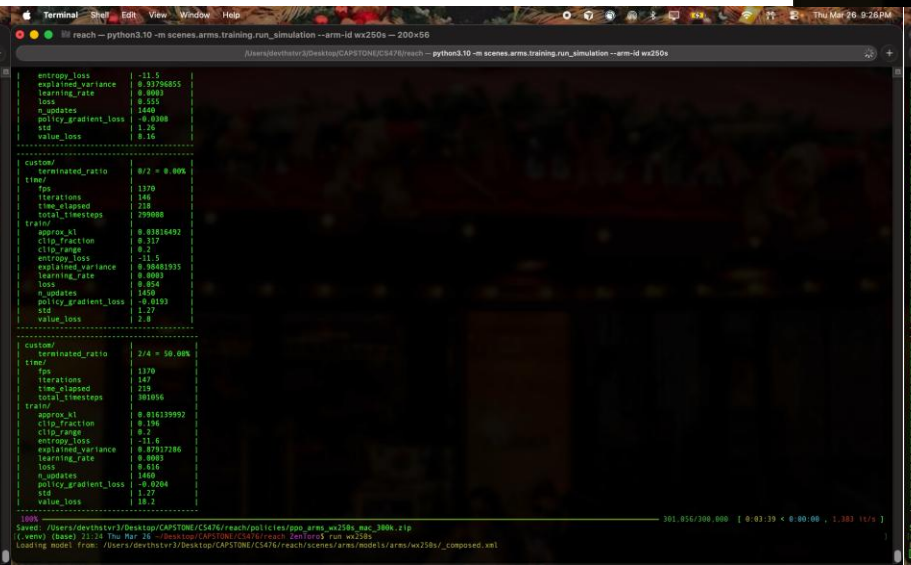
Drop-in robots - Standard folder layout + auto-discovery; train/run with one arm_id.

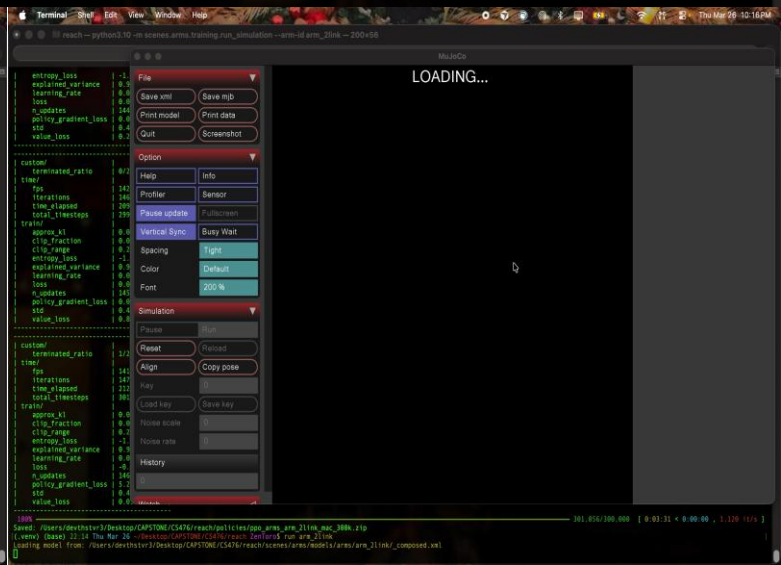
YAML-driven - Rewards, steps, and run settings without editing core code.

Same workflow everywhere - Add model → configure → train / run / evaluate (CLI).



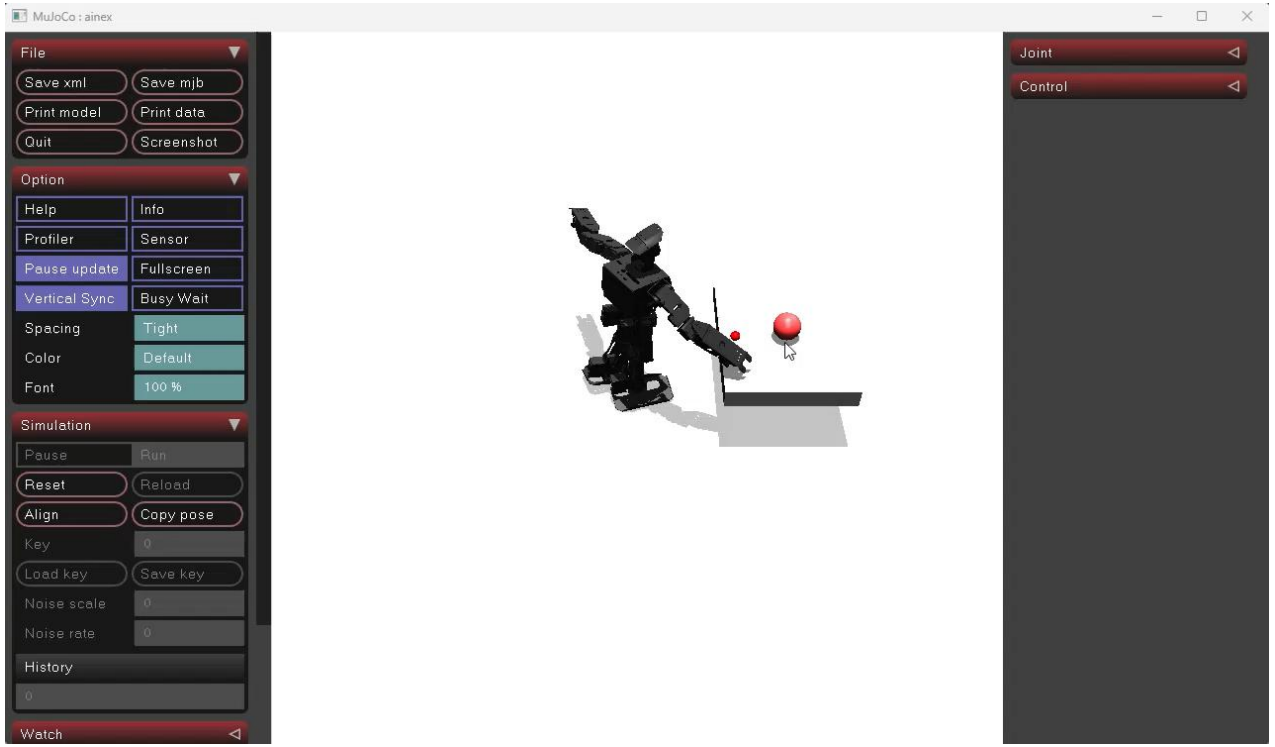






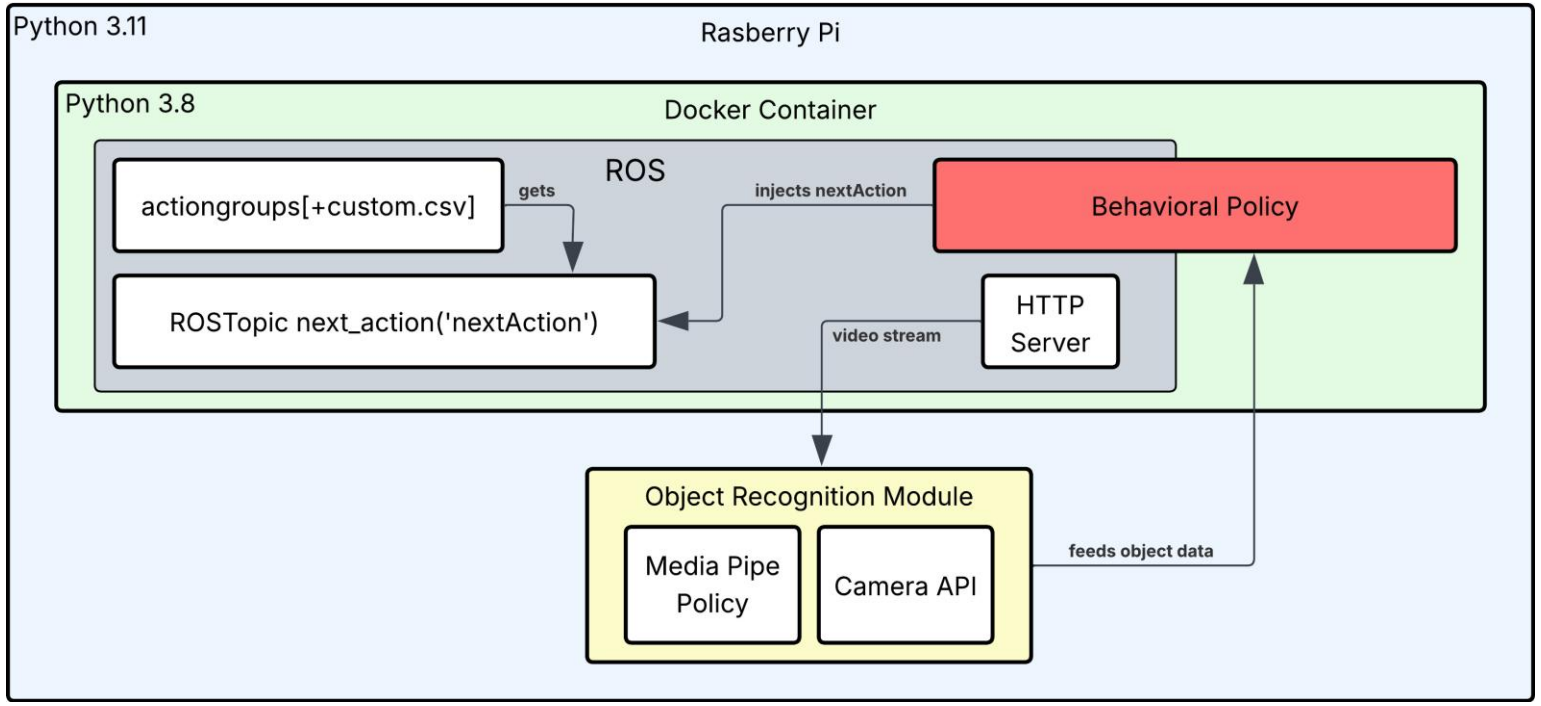
AINex Reaching Task Improvement

- Involved all degrees of freedom in the reaching task
- Improved reward function
- Enhanced Ball Physics

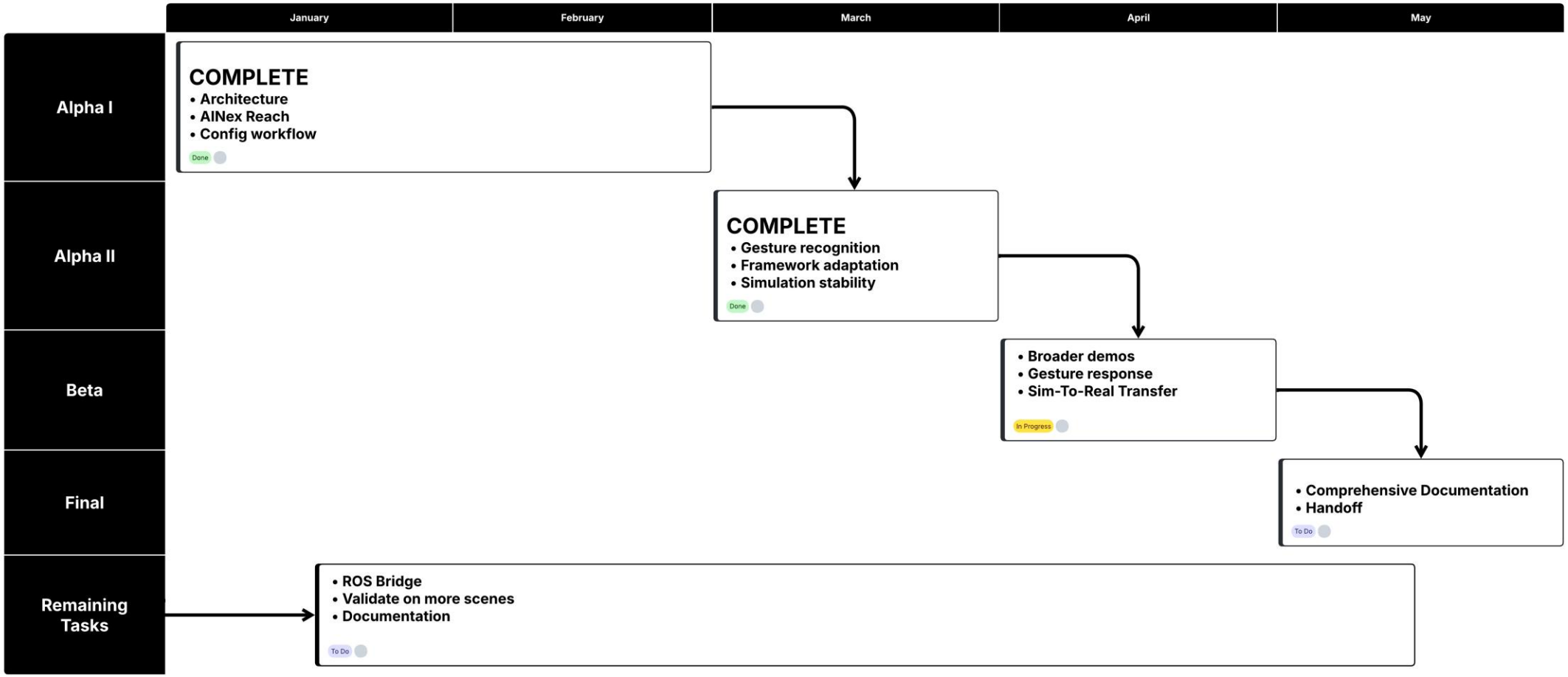


Challenges & Resolutions

- Onboard Raspberry Pi had conflicting Python versions and Docker Constraints
  - Built local HTTP Server for access to video stream
- Handshake images for gesture recognition training are two-handed
  - We plan on creating our own image set for training



Plan & Schedule



Conclusion

Built: robust reaching + adaptable framework + gesture recognition

Proved: one reproducible YAML workflow across tasks

Delivered value: faster research iteration for biomechatronics

Next: acceptance demo with generalization and sim-to-real readiness

Thank You